

Python Lists and Dictionaries - Complete Q&A; Notes

Q1. What is Labelled Indexing? Can Lists and Tuples Use It?

Lists and tuples support positional indexing only (0,1,2...). Labelled indexing is provided by dictionaries through keys. NamedTuple can provide label-like access for tuples.

Q2. Where is a Dictionary Best Used?

Dictionaries are best for key-value data such as student records, employee records, contacts, settings, API responses, and frequency counting.

Q3. Can a Tuple Inside a List Be Updated?

Tuples are immutable, so their elements cannot be changed directly. However, a list can replace an entire tuple because lists are mutable.

Q4. Which is Faster: List or Dictionary?

It depends on the operation. Lists are fast for index access. Dictionaries are fast for key lookup because they use hashing.

Q5. Why is a Dictionary Faster Than a List for Searching?

A list may require scanning elements one by one (linear search). A dictionary uses hashing to locate a key directly, giving average $O(1)$ lookup time.

Q6. What Happens During Insertion in a List?

Lists are dynamic arrays. If free capacity exists, the element is placed in the next slot. If capacity is full, Python allocates a larger array and copies elements.

Q7. What Happens During Insertion in a Dictionary?

Python computes a hash for the key, finds a bucket in the hash table, and stores the key-value pair there.

Q8. Compare List and Dictionary: Creation, Insertion, Deletion, Searching, Traversal

Creation: $O(n)$ for both. Insert at end: $O(1)$ average for both. Insert at beginning: List $O(n)$, Dictionary $O(1)$ average. Delete: List $O(n)$ in many cases, Dictionary $O(1)$ average by key. Search: List $O(n)$, Dictionary $O(1)$ average. Traversal: $O(n)$ for both.

Q9. Why is Insert at Beginning Slow in a List?

All existing elements must be shifted one position to the right before inserting the new element.

Q10. What Does 'Dictionary is an Ordered Collection' Mean?

Since Python 3.7+, dictionaries preserve insertion order. Keys appear in the order they were inserted. Ordered does not mean sorted.

Q11. How Can the Attendance Program Be Implemented with a Dictionary?

Use day labels as keys (Day1, Day2, etc.) and attendance status (P/A) as values. This makes the code more readable and realistic.

Q12. Which Searching Algorithm is Used in a Python Dictionary?

Python dictionaries use Hashing (Hash Table Lookup) for searching, insertion, and deletion.

Q13. Which Algorithm Was Used Before and After Python 3.6?

Both before and after Python 3.6, dictionaries used Hashing with Open Addressing. Python 3.6 introduced a compact dictionary layout that preserved insertion order.

Time Complexities Comparison Table

Operation	List	Dictionary	Reason
Creation	$O(n)$	$O(n)$	All elements/pairs must be stored
Insert at End	$O(1)$ avg	$O(1)$ avg	Direct append / hash insertion
Insert at Beginning	$O(n)$	$O(1)$ avg	List shifts elements; dict doesn't
Search	$O(n)$	$O(1)$ avg	Linear scan vs hashing
Delete	$O(n)$	$O(1)$ avg	Shifting vs hash lookup
Traversal	$O(n)$	$O(n)$	Must visit all items
Index Access	$O(1)$	N/A	Direct array indexing
Key Access	N/A	$O(1)$ avg	Hash table lookup

Why is Dictionary Search Faster?

Dictionary uses hashing and direct bucket lookup, while a list may scan elements one by one.

What Does Ordered Dictionary Mean?

Python 3.7+ preserves insertion order of keys.

Which Searching Algorithm is Used in Dictionary?

Hashing (Hash Table Lookup).

Before and After Python 3.6 Dictionary Insertion

Both use hashing with open addressing. Python 3.6 introduced compact dictionaries preserving insertion order.